Synthetic Data Sub-Function Standup for Quality Engineering (Operating Model, Pods & KPIs)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

- What this is . . . 4
- The target model . . . 5
- How it works — the operating model . . . 5
- Benefits . . . 6
- Limitations . . . 6
- Who operates it . . . 6

Requirements 6

- S1. An executive sponsor and a funded charter . . . 7
- S2. Defined pod structure and named roles . . . 7
- S3. An intake and prioritization scheme . . . 7
- S4. A defined KPI framework with baselines . . . 8
- S5. A reference-architecture definition of record . . . 8
- S6. A maturity target and cadence . . . 9

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

- 1. A Python data/engineering environment . . . 9
- 2. A target clinical data model / standard . . . 9
- 3. An isolated non-production test environment . . . 10
- 4. Privacy, governance & compliance stakeholders and policy . . . 10
- 5. AI/QA use cases and a pilot scope . . . 10
- 6. Reference/real-data access for fidelity (read-only, minimized) . . . 11

Implementation 11

- Artifacts you will produce (naming register) . . . 11
- Stage 1 — Assess . . . 12
- Stage 2 — Design . . . 12
- Stage 3 — Generate/Build . . . 13
- Stage 4 — Validate . . . 14
- Stage 5 — Expand . . . 14

Testing & Validation 15

- A. Operating-model validation (run at standup and on review cadence) . . . 15
- B. Per-dataset validation (run at Gate D for every dataset) . . . 16

Troubleshooting 17

- Standup / operating model . . . 17
- Generation / build . . . 18

Validation / privacy . . . 19
KPIs / measurement . . . 19
Best Practices 20
Operating model . . . 20
Engineering . . . 20
Validation & quality . . . 21
Privacy & safety (non-negotiable) . . . 21
Maturity & scaling . . . 21
Deliverables Summary 21
Documentation . . . 22
The 5 Python tools (`SYNFUNC-Scripts.zip`) . . . 22
Appendix A — PowerShell Scripts 24

Executive Summary

Outcome. This solution stands up a **synthetic data sub-function** inside quality engineering and turns it into a **repeatable, measured service** — one that produces trustworthy synthetic clinical data and simulation on demand for AI and QA teams across programs, and proves it is trustworthy with evidence rather than assurances.

The problem it solves. Building this capability is an **operating-model** problem, not just a tooling one. Teams can run a generator once; few can turn synthetic data into a service with clear ownership, a predictable front door, quality gates, and metrics that survive executive review. Without that structure, synthetic-data work is heroic, inconsistent, and unmeasured — and quietly risky.

The approach. The solution supplies the missing structure end to end: the **team design** (pods/squads behind a single intake queue), **intake and prioritization**, a **reference architecture** that keeps engines (Synthea, commercial platforms, AI frameworks) swappable behind a stable interface, a **maturity model** with a review cadence, and a **KPI framework** that tracks both delivery quality (validation pass rate, defect escape, rework, gate cycle time) and engineering productivity (lead time, throughput, flow efficiency, WIP). It is delivered as self-guided documentation plus **five working Python tools** that operationalize the model — a maturity self-assessment, an intake/capacity analyzer, a fidelity/coverage report, a privacy screen, and a KPI rollup with a reference-architecture validator. Everything follows the house model **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback.

Why it is safe. The sub-function operates only on **synthetic (artificial) data in isolated non-production environments**. **No real PHI is ever placed into a shared synthetic harness, twin, or AI test environment** — the reference architecture enforces that boundary and the tools are safe-by-default (dry-run to write, never write to shared locations, never emit real values). Privacy gates produce **technical evidence and a pass/fail signal**; the regulatory and legal **determinations** — HIPAA/HITRUST de-identification, IRB/privacy-office sign-off, disclosure-risk acceptance — are explicitly **deferred to the organization's privacy, compliance, and legal functions**.

Bottom line. A synthetic-data capability that operates as a repeatable, measured service — structured teams, clear intake, and KPIs proving delivery quality — serving AI and QA across programs, at a fraction of the cost of a bespoke consulting standup.

Solution Overview

What this is

This solution stands up a **synthetic data sub-function** inside a greenfield (or immature) quality-engineering organization. The problem it solves is an **operating-model** problem, not merely a tooling one: many teams can run a generator once, but few can turn synthetic data and simulation into a **repeatable, measured service** that serves AI, QA, and product teams across programs. This solution supplies the missing structure — the team design (pods/squads), the intake and prioritization flow, a reference architecture, a maturity model, and a **KPI framework** that tracks both delivery quality and engineering productivity — and packages five Python tools that operationalize that model.

The sub-function's mission is simple to state: **produce trustworthy synthetic clinical data and simulation on demand, safely, and prove it is trustworthy with evidence.** "Trustworthy" means it is fit for the consuming AI/QA use case (fidelity and coverage) and it is safe to share (privacy), and "prove it" means every dataset ships with a validation record, not a promise.

Everything here operates on **synthetic (artificial) data in isolated non-production environments**. **No real Protected Health Information (PHI) is ever placed into a shared synthetic harness, twin, or AI test environment.** Where real data must inform fidelity, it is used read-only, minimized, and only under the organization's governing privacy controls (see `02-requirements.md` and the shared `global-requirements-synthetic.md`).

The target model

The sub-function is **model-agnostic but standard-anchored.** It generates and validates against a **target clinical data model** chosen with the consuming teams — typically **FHIR R4**, the **OMOP CDM**, or the source system's native schema — and it records that choice as a first-class artifact so fidelity, coverage, and privacy are all measured against a fixed target. The operating model does not assume one generator: **Synthea**, commercial synthetic-data platforms, and AI/model frameworks are integrated at the pattern level (an "engine" behind a stable internal interface), so the sub-function can adopt or swap engines without re-architecting intake, validation, or KPIs. Vendor console internals are cited to the vendor's documentation or flagged "verify against your version" — never invented.

How it works — the operating model

The sub-function runs as a small number of **pods** behind a single **intake queue**:

• **Intake pod** — owns the request queue, triage, scoping, and prioritization. Every request enters as a scoped **Intake Request** and leaves as a committed, sized unit of work or a documented decline.

• **Generation pod(s)** — build and run the synthetic-data / twin engines against the target model, producing named datasets.

• **Validation & governance pod** — runs the fidelity, coverage, and privacy gates, produces the validation record, and holds the release decision (technical pass/fail; the privacy/legal *determination* is deferred to those functions).

• **Platform/enablement pod** (may be virtual at low maturity) — owns the reference architecture, shared libraries, environments, and the five tools.

Work flows through the house deployment model used by every solution in this category:

Assess → **Design** → **Generate/Build** → **Validate** → **Expand**, pilot-first, with explicit pass/fail (fidelity/privacy) gates and a documented rollback.

• **Assess** — baseline the current capability with the maturity self-assessment; agree sponsorship, environment, governance stakeholders, target model, and pilot AI use cases.

• **Design** — define the pods, the intake flow, the reference architecture, the KPI framework, and the pilot's pass/fail gates. Produce the **Capability Charter**.

• **Generate/Build** — stand up the environment and engines; run the pilot request end-to-end, producing a named dataset (`Synthetic EHR – Pilot – v1`).

• **Validate** — run fidelity, coverage, and privacy gates; produce the validation record; make the release decision at the gate.

• **Expand** — onboard more request types, programs, and engines; advance a maturity level; grow the KPI baseline into a trend.

Benefits

• **Repeatable service, not heroics** — a stable intake-to-release flow so any competent engineer can run a request the same way, with the same evidence.

• **Measured** — a KPI framework that makes delivery quality (defect escape, validation pass rate, rework) and engineering productivity (lead time, throughput, flow efficiency) visible and trended.

• **Safe by construction** — isolation, privacy gates, and safe-by-default tooling mean disclosure risk is bounded and evidenced, with the *determination* deferred to privacy/compliance/legal.

• **Scalable** — a maturity model and reference architecture give a clear path from an ad-hoc script to a multi-program platform, and engine-agnostic patterns avoid vendor lock-in.

Limitations

• This is an **operating-model and enablement** solution. It gives you the structure, the gates, and five working Python tools; it does **not** ship a production data platform or a specific vendor license.

• **Fidelity claims require a reference** (governed real statistics or agreed published distributions). Without one, coverage and structural checks still run, but distributional-fidelity claims are unfounded — the tools will say so.

• **Regulatory and legal determinations are out of scope** — HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, and disclosure-risk acceptance belong to those functions. The sub-function supplies method and evidence, not the ruling.

• The KPI framework is only as good as the intake discipline behind it: garbage-in intake produces meaningless metrics. The tools assume requests are logged.

Who operates it

A competent data/platform engineer can run every step and every tool here. Specialist input (clinical SME for plausibility, privacy officer for disclosure-risk threshold) is called out where the step needs it.

Requirements

This solution builds **on** the six category-wide requirements in `global-requirements-synthetic.md` (Python data environment; target clinical data model; isolated non-production environment; privacy/governance stakeholders and policy; AI/QA use cases and a pilot scope; read-only minimized reference data for fidelity). **Verify those six first** — they are not repeated here.

Below are the **solution-specific** prerequisites for standing up the sub-function as an operating model. Each is stated as: **Description / Why required / How to verify / How to implement if missing.**

> ***Boundary reminder.*** *Never place real PHI into a shared synthetic harness, twin, or AI test environment. Regulatory/legal determinations (de-identification, IRB, disclosure-risk acceptance) are deferred to the organization's privacy, compliance, and legal functions.*

S1. An executive sponsor and a funded charter

• **Description:** A named **executive sponsor** in (or over) quality engineering who owns the sub-function's mandate, funds at least a pilot's worth of capacity, and can arbitrate cross-program prioritization.

• **Why required:** A shared service that cannot say "no" or "not yet" to a program will be captured by the loudest requester; a standup without funded capacity produces a charter no one honors. The sponsor is the escalation point the intake pod needs.

• **How to verify:** Confirm a named sponsor, a funding line for at least the pilot, and written authority to prioritize across programs. Recorded in the **Capability Charter – Sub-Function – v1**.

• **How to implement if missing:** Run the maturity self-assessment (Assess stage) and use its output to make the sponsorship ask concrete (scope, capacity, first pilot value) before designing pods.

S2. Defined pod structure and named roles

• **Description:** A documented **pod/squad structure** (Intake, Generation, Validation & Governance, Platform/enablement — some may be virtual at low maturity) with **named people or committed FTE fractions** and a RACI for the intake-to-release flow.

• **Why required:** Pods and a single intake queue are what make the service repeatable; without named ownership, requests route by relationship, not by process, and KPIs cannot be attributed.

• **How to verify:** Confirm each pod has an owner and committed capacity, and that the RACI names who triages, who generates, and who holds the release decision. Use `intakequeue.py --capacity` to model whether the committed capacity covers the expected intake.

• **How to implement if missing:** Start with a minimum viable structure (one intake owner, one generation engineer, one validation owner — the last two may be the same person at pilot) and grow as maturity advances.

S3. An intake and prioritization scheme

• **Description:** A defined **intake form** (fields: requesting program, AI/QA use case, target model/domains, data volume, fidelity need, privacy sensitivity, needed-by date) and a **prioritization method** (a documented scoring model — value, urgency, effort, risk).

• **Why required:** Intake is the front door of the operating model and the source of every KPI. An undefined intake produces unscoped work and meaningless lead-time metrics.

• **How to verify:** Confirm the intake form exists and the scoring weights are agreed with the sponsor. Validate a queue export with `intakequeue.py`.

• **How to implement if missing:** Adopt the intake schema this solution's `intakequeue.py` expects as a starting point and tune the weights with the sponsor.

S4. A defined KPI framework with baselines

• **Description:** An agreed set of **delivery-quality** KPIs (e.g. validation pass rate, defect escape rate, rework rate, gate cycle time) and **engineering-productivity** KPIs (e.g. request lead time, throughput, flow efficiency, WIP), each with a **definition, a target, and a baseline** taken at standup.

• **Why required:** "Repeatable, measured service" is the whole point; without agreed definitions and a baseline, later numbers cannot show improvement and cannot survive review.

• **How to verify:** Confirm each KPI has an owner, a formula, a target, and a baseline value. Compute a rollup with `kpirollup.py`.

• **How to implement if missing:** Adopt the KPI catalog this solution defines (see `03-implementation.md`) and take the baseline on the pilot before expanding.

S5. A reference-architecture definition of record

• **Description:** A written **reference architecture** naming the environment tiers (isolated non-prod), the engine interface (so Synthea / commercial platform / AI framework are swappable behind it), the storage and access-control model, the validation/gate stage, and the data-flow boundaries (what may and may not cross into the harness).

• **Why required:** Engine-agnostic patterns and a fixed gate stage are what let the sub-function scale and swap tools without re-architecting; the architecture is also where the **no-real-PHI-in-shared- harness** boundary is enforced.

• **How to verify:** Confirm the architecture document exists and lists the required tiers, the engine interface, and the isolation boundary. Validate a machine-readable config with `kpirollup.py --refarch`.

• **How to implement if missing:** Use the reference-architecture config schema the validator (`kpirollup.py --refarch`) expects as the starting template and fill it in with the platform pod.

S6. A maturity target and cadence

• **Description:** An agreed **maturity model** (levels from ad-hoc to optimized across dimensions: operating model, engineering, validation/quality, privacy/governance, and adoption) and a **review cadence** at which the sub-function re-assesses and advances.

• **Why required:** Standup is stage one of five (Assess→…→Expand); without a maturity target and cadence, the sub-function plateaus at pilot and never becomes a program-wide service.

• **How to verify:** Confirm the current level is baselined and a target level + review date are set. Run `maturityassess.py` to capture the baseline.

• **How to implement if missing:** Run the maturity self-assessment at standup to set the baseline and the first target.

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate* ***synthetic*** *(artificial) data and run in* ***isolated non-production*** *test environments.* ***Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment*** *— where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls.* ***Synthetic-data and digital-twin tools*** *(e.g. Synthea, and commercial platforms) and* ***AI/model frameworks*** *are integrated at the pattern/architectural level; specific vendor console internals are* ***cited or deferred*** *to that vendor's documentation, never invented.* ***Regulatory and legal interpretation*** *(HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is* ***deferred to the organization's privacy, compliance, and legal functions*** *— the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

This is the ordered standup procedure. It follows the house model **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback. Every artifact is **named** with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in later stages, testing, troubleshooting, and the tools.

> ***Hard boundaries throughout.*** *No real PHI enters a shared harness/twin/AI test environment. Any reference to real data is read-only, minimized, and under the governing privacy controls. Regulatory/legal determinations are deferred to privacy/compliance/legal.*

Artifacts you will produce (naming register)

Artifact	Named instance (pilot)	Produced in
Capability baseline	`Maturity Baseline – Sub-Function – v1`	Assess
Sponsorship & mandate	`Capability Charter – Sub-Function – v1`	Design
Pod & RACI design	`Pod Design – Sub-Function – v1`	Design
Intake definition	`Intake Scheme – Sub-Function – v1`	Design
Reference architecture	`Reference Architecture – Sub-Function – v1`	Design
KPI framework	`KPI Framework – Sub-Function – v1`	Design

Artifact	Named instance (pilot)	Produced in
Pilot dataset	`Synthetic EHR – Pilot – v1`	Generate/Build
Validation record	`Validation Record – Pilot – v1`	Validate
Expansion plan	`Expansion Plan – Sub-Function – v1`	Expand

Stage 1 — Assess

Goal: an honest baseline and a concrete sponsorship ask.

1. **Confirm the six global requirements** (`global-requirements-synthetic.md`) and the six solution-specific ones (`02-requirements.md`). Record gaps; do not proceed past Design with S1 (sponsor) or the isolated environment unresolved.

1. **Run the maturity self-assessment.** Execute `maturityassess.py` with your dimension scores to produce `Maturity Baseline – Sub-Function – v1` (a JSON + printed summary). This baseline is reused as the KPI baseline anchor and the Expand-stage comparison.

1. **Confirm the pilot inputs** with the consuming AI/QA teams: the **target clinical data model** (FHIR / OMOP / native), the in-scope domains (patients, encounters, conditions, meds, labs, vitals), the pilot AI/QA use case, and the **reference source** for fidelity (governed real stats or agreed published distributions).

1. **Make the sponsorship ask** using the baseline: scope, pilot capacity, first pilot value. Obtain a named sponsor and funded pilot capacity (S1).

Gate A (Assess→Design): sponsor named + funded pilot capacity + isolated non-prod environment confirmed + target model chosen. If any fail, stop; the standup is not ready.

Stage 2 — Design

Goal: the operating model on paper, ready to run.

1. **Author** `Capability Charter – Sub-Function – v1` — mission, scope (what the sub-function does and explicitly does *not* do), the sponsor and escalation path, and the deferral of regulatory/legal determinations to privacy/compliance/legal.

1. **Design the pods** → `Pod Design – Sub-Function – v1`. Define Intake, Generation, Validation & Governance, and Platform/enablement pods with owners and committed FTE fractions, plus a RACI for the intake-to-release flow. **Model the capacity** with `intakequeue.py --capacity` against expected intake; adjust FTE or scope until the model is not structurally over-committed.

1. **Define intake** → `Intake Scheme – Sub-Function – v1`. Adopt the intake form fields and the

prioritization scoring model (value, urgency, effort, risk with agreed weights). This is the schema `intakequeue.py` consumes.

1. **Define the reference architecture** → `Reference Architecture – Sub-Function – v1`. Specify

the isolated non-prod tiers, the **engine interface** (Synthea / commercial platform / AI framework swappable behind it), storage + access control, the fixed **validation/gate stage**, and the data-flow boundary that enforces no-real-PHI-in-shared-harness. Capture it as the config `kpirollup.py --refarch` validates.

1. **Define the KPI framework** → `KPI Framework – Sub-Function – v1`. Adopt the catalog below;

give each KPI a definition, target, owner, and baseline slot.

Delivery-quality KPIs

- *Validation pass rate* — % of datasets passing all gates on first submission.
- *Defect escape rate* — issues found by consumers ÷ total issues (lower is better).
- *Rework rate* — % of requests requiring a second generation/validation cycle.
- *Gate cycle time* — median time in the validation stage.

Engineering-productivity KPIs

- *Request lead time* — intake-accepted to release (median + p85).
- *Throughput* — requests released per period.
- *Flow efficiency* — active time ÷ total lead time.
- *WIP* — mean requests in progress (watch against pod capacity from step 6).

Gate B (Design→Build): charter, pod design, intake scheme, reference architecture (validated by `kpirollup.py --refarch`), and KPI framework all exist and are agreed. Reference architecture config must pass `kpirollup.py --refarch` with **no ERROR-level findings**.

Stage 3 — Generate/Build

Goal: the environment and engine stood up; one pilot request run end-to-end.

1. **Stand up the isolated environment and engine** per `Reference Architecture – Sub-Function – v1`.

Install the Python data environment; wire the first engine (e.g. Synthea or a commercial platform) behind the engine interface. Verify the environment is not connected to production PHI stores.

1. **Accept the pilot request through intake.** Log it in the queue; confirm `intakequeue.py`

prioritizes and reports it cleanly.

1. **Profile the target/reference** with `datasetprofile.py` against the agreed reference statistics

(read-only, minimized) to establish the fidelity comparison baseline for the target domains.

1. **Generate the pilot dataset** → `Synthetic EHR – Pilot – v1` against the chosen target model

and in-scope domains, written **only** to the isolated environment. (If your engine is Synthea or a commercial platform, run it per that vendor's documentation; treat its output as the named dataset.)

Gate C (Build→Validate): `Synthetic EHR – Pilot – v1` exists in the isolated environment, conforms structurally to the target model, and contains the in-scope domains. No real PHI present.

Stage 4 — Validate

Goal: evidence the dataset is fit and safe; a release decision at the gate.

1. **Run the fidelity/coverage report.** Execute `datasetprofile.py` comparing

`Synthetic EHR – Pilot – v1` to the reference baseline from step 12; produce the coverage and distributional-fidelity section of the validation record.

1. **Run the privacy/validation check.** Execute `privacycheck.py` on the dataset to screen for

direct-identifier leakage, uniqueness/quasi-identifier risk signals, and exact copies of reference rows. This produces **technical evidence and a pass/fail signal only** — the disclosure-risk **determination** is deferred to the privacy officer.

1. **Assemble** `Validation Record – Pilot – v1` — the fidelity/coverage results, the privacy screen

results, the gate decision, and the named reviewers.

Gate D (the release gate — pass/fail):

• **Fidelity/coverage:** meets the pilot's agreed thresholds (coverage of in-scope domains ≥ target;

distributional distance within tolerance where a reference exists).

• **Privacy:** `privacycheck.py` reports **no direct identifiers**, **no exact reference-row copies**,

and quasi-identifier risk **below the agreed threshold**; privacy officer accepts the evidence.

• **Decision:** PASS → release to the consuming AI/QA use case. FAIL → **rollback**: quarantine

`Synthetic EHR – Pilot – v1` in the isolated environment, record the failing gate in the validation record, and return the request to the Generation pod for a second cycle (counts against *rework rate*). Never release on a failed privacy gate.

Stage 5 — Expand

Goal: turn the pilot into a scaling, measured service.

1. **Baseline the KPIs.** Compute the first `KPI Framework – Sub-Function – v1` rollup with

`kpirollup.py` from the intake/validation data of the pilot. This is the trend anchor.

1. **Onboard the next request types and programs** through the same intake and gates; add engines

behind the engine interface as needed (re-run `kpirollup.py --refarch` on any architecture change).

1. **Re-run the maturity self-assessment** (`maturityassess.py`) on cadence, compare to

`Maturity Baseline – Sub-Function – v1`, and record the next target in `Expansion Plan – Sub-Function – v1`.

1. **Review KPIs on cadence** with the sponsor; watch WIP against pod capacity (`intakequeue.py --capacity`) and

re-scope or re-staff before lead time degrades.

Steady state: every request enters through `Intake Scheme – Sub-Function – v1`, produces a named dataset, ships with a `Validation Record`, and contributes to the KPI trend — a repeatable, measured service.

Testing & Validation

These procedures validate both the **operating model** (is the sub-function set up correctly?) and each **dataset** it produces (is the output fit and safe?). Each test states its **expected pass/fail signal**. Run the operating-model tests once at standup and on cadence; run the per-dataset tests at the Validate gate (Gate D) for every request.

> *No real PHI in a shared harness/twin/AI test environment. Privacy tests produce **technical evidence**; the disclosure-risk **determination** is the privacy officer's.*

A. Operating-model validation (run at standup and on review cadence)

A1. Maturity baseline is captured

- **Procedure:** Run `maturityassess.py` with the agreed dimension scores.
- **Pass:** Produces `Maturity Baseline – Sub-Function – v1` with a level per dimension and an overall

level; the printed summary matches the recorded charter target.

- **Fail:** Missing dimensions or an unset target level → complete the assessment before Design closes.

A2. Reference architecture config is valid

- **Procedure:** Run `kpirollup.py --refarch --config <arch>.json` against

`Reference Architecture – Sub-Function – v1`.

- **Pass:** Exit code 0, **no ERROR findings** — required tiers present, an engine interface defined,

the isolation/no-real-PHI boundary asserted, and a validation/gate stage present.

- **Fail:** Any ERROR finding (e.g. missing isolation boundary, no gate stage) → fix the architecture;

Gate B cannot close.

A3. Intake queue is well-formed and prioritized

- **Procedure:** Run `intakequeue.py --input <queue>.csv` on the current queue.
- **Pass:** All rows parse; each request gets a priority score and rank; capacity-vs-demand summary

prints. No rows rejected for missing required fields.

- **Fail:** Rejected rows or unscored requests → correct the intake data/schema.

A4. Pod capacity covers expected demand

- **Procedure:** Run `intakequeue.py --capacity` with pod FTE and expected request mix.
- **Pass:** Modeled utilization for each pod ≤ the agreed ceiling (default 85%); no pod structurally over-committed.
- **Fail:** Any pod > ceiling → re-staff or re-scope before committing the charter.

A5. KPI rollup computes against definitions

- **Procedure:** Run `kpirollup.py` on the pilot's intake + validation records.
- **Pass:** Every KPI in `KPI Framework – Sub-Function – v1` computes to a number with its baseline recorded; delivery-quality and productivity sections both populate.
- **Fail:** A KPI cannot compute (missing timestamps/fields) → fix the source data capture in intake.

B. Per-dataset validation (run at Gate D for every dataset)

Applied to the pilot as `Synthetic EHR – Pilot – v1`; applied by name to every later dataset.

B1. Structural / schema conformance

- **Procedure:** `datasetprofile.py --dataset <data> --mode structure` against the target model.
- **Pass:** All in-scope domains present; required fields typed correctly; referential links (patient→encounter→observation) resolve.
- **Fail:** Missing domain or broken link → regenerate (counts as rework).

B2. Coverage

- **Procedure:** `datasetprofile.py --dataset <data> --mode coverage` against the pilot's coverage targets (value sets / categories that must appear).
- **Pass:** Coverage of in-scope categories ≥ the agreed target (default ≥ 90% of required categories represented).
- **Fail:** Below target → extend scenarios/generation and re-run.

B3. Distributional fidelity (requires a reference)

- **Procedure:** `datasetprofile.py --dataset <data> --reference <ref-stats> --mode fidelity`.
- **Pass:** Distributional distance per compared field within tolerance (default: normalized distance ≤ 0.10 or as agreed). **If no reference exists, this test is reported as N/A — not passed** — and no distributional-fidelity claim is made.

- **Fail:** Distance beyond tolerance → tune the engine/parameters and re-run.

B4. Privacy — direct-identifier screen

- **Procedure:** `privacycheck.py --dataset <data> --check identifiers`.
- **Pass:** No direct identifiers (names, MRNs, SSNs, contact fields, dates-of-birth in identifying form) present in the synthetic output.
- **Fail:** Any direct identifier → **block release**; investigate the engine/config leak.

B5. Privacy — reference-copy / memorization screen

- **Procedure:** `privacycheck.py --dataset <data> --reference <ref> --check copies`.
- **Pass:** No synthetic record is an exact (or near-exact) copy of a reference record.
- **Fail:** Any exact copy → **block release**; the engine is leaking reference rows.

B6. Privacy — quasi-identifier / uniqueness risk signal

- **Procedure:** `privacycheck.py --dataset <data> --check uniqueness --k <k>`.
- **Pass:** Share of records that are unique on the quasi-identifier set is **below the agreed threshold** (default: k-anonymity with k ≥ agreed value; report the fraction violating it).
- **Fail / escalate:** Above threshold → escalate the evidence to the privacy officer for the disclosure-risk **determination**; do not self-clear.

Gate D result: all B1–B6 must PASS (or fidelity N/A with explicit acknowledgement) **and** the privacy officer must accept B4–B6 evidence before `Synthetic EHR – Pilot – v1` is released. Any privacy fail is a hard block with rollback per `03-implementation.md`.

Troubleshooting

Common failure modes when standing up and operating the synthetic-data sub-function, and how to resolve them. Grouped by stage. Named artifacts and tools follow `03-implementation.md`.

> *Any privacy failure is a **hard block** with rollback — never "troubleshoot around" it by releasing. Regulatory/legal determinations stay with privacy/compliance/legal.*

Standup / operating model

Requests keep arriving outside the intake queue.

- *Cause:* No enforced single front door; requesters use relationships.
- *Fix:* Route every request — including sponsor "favors" — through `Intake Scheme – Sub-Function – v1`.

Log expedites as priority overrides so they still appear in `intakequeue.py` and the KPIs. If it is not in the queue, it does not get done.

`intakequeue.py` **rejects rows.**

- *Cause:* Missing required fields or out-of-range scores (value/urgency/effort/risk must be 1–5).

• *Fix:* Correct the intake data to match the schema (see `REQUIRED_FIELDS` in the tool). A rejected row is unscoped work — do not let it silently proceed.

A pod shows OVER-COMMITTED (or NO-CAPACITY) in `intakequeue.py --capacity`.

• *Cause:* Committed FTE cannot cover the demand mix at the utilization ceiling (default 85%).

• *Fix:* Re-staff, re-scope, or defer lower-priority requests before committing the charter. Do not commit to demand the capacity model rejects; that is how lead time silently degrades.

`kpirollup.py --refarch` reports ERROR findings.

• *Cause:* The reference-architecture config is missing a required element — most often the `isolation_boundary` assertion, a defined `engine_interface`, or the `gate_stage`.

• *Fix:* Complete the config. The **no-real-PHI isolation boundary** and a present gate stage are hard requirements; Gate B cannot close with any ERROR finding.

Maturity level looks stuck at 1–2 despite good tooling.

• *Cause:* `maturityassess.py` caps overall maturity by the weakest of validation/quality and privacy/governance — strong engineering cannot compensate for missing gates.

• *Fix:* Raise the gate-critical dimensions (stand up the validation record and privacy screens) before expecting the overall level to move.

Generation / build

Generated dataset fails structural conformance (B1).

• *Cause:* Engine output does not match the target model (missing domain, broken patient→encounter→observation links).

• *Fix:* Correct the engine mapping/config to the target model; regenerate `Synthetic EHR – Pilot – v1` (counts as rework). If the engine is Synthea or a commercial platform, verify the mapping against that vendor's documentation for your version.

Coverage below target (B2).

• *Cause:* Scenario set or generation parameters do not exercise all required categories/value sets.

• *Fix:* Extend the scenario suite or generation parameters to include the missing categories reported by `datasetprofile.py --mode coverage`, then re-run.

Engine swap broke the pipeline.

• *Cause:* Vendor-specific behavior leaked outside the engine interface.

• *Fix:* Keep all engine specifics behind the reference-architecture engine interface; the intake, validation, and KPI stages must not know which engine ran. Re-validate the architecture with `kpirollup.py --refarch` after any engine change.

Validation / privacy

`datasetprofile.py` **fidelity shows N/A.**

- *Cause:* No `--reference` supplied — expected when no governed reference or agreed distribution exists.
- *Fix:* This is **not** a pass and **not** a failure: report distributional fidelity as N/A and make no fidelity claim. Coverage and structure still gate. Obtain a governed/minimized reference (or agree published distributions) if a fidelity claim is required.

Fidelity distance is out of tolerance (B3).

- *Cause:* Synthetic distributions diverge from the reference beyond the agreed tolerance.
- *Fix:* Tune engine parameters (prevalence, age/sex mix) and re-run. Do not loosen the tolerance to force a pass without sponsor agreement.

`privacycheck.py` **reports HARD FAIL on identifiers (B4).**

- *Cause:* A direct-identifier field name or a value matching an SSN/email/phone pattern is present.
- *Fix:* **Block release.** Find and remove the leak in the engine/config; direct identifiers must never appear in synthetic output. Regenerate and re-screen.

`privacycheck.py` **reports HARD FAIL on copies (B5).**

- *Cause:* Synthetic rows are exact copies of reference rows — the engine is memorizing/leaking real data.
- *Fix:* **Block release.** Investigate the engine's use of reference data (it should learn distributions, not copy rows). This is a serious safety issue; fix before any further generation.

`privacycheck.py` **reports ESCALATE on uniqueness (B6).**

- *Cause:* Too many records are unique on the quasi-identifier set (below the agreed k).
- *Fix:* Escalate the evidence to the **privacy officer** for the disclosure-risk determination — do not self-clear. Mitigations (generalization, suppression, larger cohorts) are applied only with the privacy officer's direction.

Everything passes but the consuming team reports the data is unusable.

- *Cause:* Coverage/fidelity gates passed but the scenarios do not match the actual AI/QA use case.
- *Fix:* Return to intake — the `use_case` scoping was incomplete. Re-scope with the consuming team; this is an intake-quality problem, and it will show up as *defect escape rate* in `kpirollup.py`.

KPIs / measurement

KPIs cannot be computed (`kpirollup.py` errors or zeros).

- *Cause:* Records are missing timestamps or validation outcomes.

• *Fix:* Ensure intake and validation capture the fields the tool expects (accepted/released times, first-pass, cycles, gate time, issues). Fix the capture, not the tool.

KPIs look good but the sponsor doesn't trust them.

• *Cause:* Side-channel work bypasses intake, so the metrics cover only part of reality.

• *Fix:* Close the side channels (see the first item). A measured service requires that *all* work flows through intake; otherwise the numbers are unrepresentative.

Best Practices

Durable practices for running the synthetic-data sub-function as a repeatable, measured service. These outlast any single engine or program.

Operating model

• **One front door.** Every request enters through `Intake Scheme – Sub-Function – v1` — no side channels. Side-channel work is invisible to the KPIs and corrodes prioritization. If the sponsor wants something expedited, expedite it *through* intake with a documented priority override.

• **Pods own outcomes, not tasks.** Keep the Validation & Governance pod's release decision independent from the Generation pod. The team that builds the dataset should not be the sole judge of whether it passes the gate.

• **Charter what you will *not* do.** The `Capability Charter` is as much about scope exclusions (production data platform, legal determinations, unbounded ad-hoc pulls) as inclusions. A shared service dies from unbounded scope.

• **Size before you commit.** Run `podcapacity.py` before accepting a wave of new demand; commit only to what utilization ≤ ceiling supports. Protect flow efficiency by capping WIP.

Engineering

• **Engine-agnostic behind an interface.** Treat Synthea, commercial platforms, and AI/model frameworks as swappable engines behind the reference architecture's engine interface. Never scatter a vendor's console specifics through your pipeline; when a step is version-specific, cite the vendor's documentation and flag "verify against your version."

• **Everything is named and versioned.** Datasets, configs, and validation records use the `<Purpose> – <Scope> – <Stage>` convention and carry a version. `Synthetic EHR – Pilot – v1` and `Synthetic EHR – Pilot – v2` must be distinguishable and traceable to their validation records.

• **Configuration as data.** Keep the reference architecture, intake scheme, and KPI definitions as

machine-readable config that the tools validate (`refarchvalidate.py`, `intakequeue.py`, `kpirollup.py`) — not as prose only. Re-validate on every change.

• **Reproducibility.** Record the engine, parameters, and seed for every dataset so a validation result can be reproduced.

Validation & quality

• **No release without a validation record.** A dataset ships with `Validation Record – <Scope> – v<n>` or it does not ship. "Looks fine" is not evidence.

• **Fidelity claims require a reference.** If there is no governed reference or agreed published distribution, report distributional fidelity as **N/A**, not as a pass. Coverage and structure still gate.

• **Trend the quality KPIs, not just the point value.** A single validation pass rate says little; the trend of pass rate, defect escape, and rework is the signal that the sub-function is maturing.

Privacy & safety (non-negotiable)

• **Never real PHI in a shared harness/twin/AI test environment.** This is architectural, not a policy sticker — the reference architecture's data-flow boundary enforces it, and `refarchvalidate.py` checks that the boundary is asserted.

• **Reference data is read-only and minimized.** When measuring fidelity against real statistics, access is read-only, minimized to the fields needed, and under the governing privacy controls.

• **Tools are safe by default.** Every script defaults to dry-run for anything that writes, never writes synthetic output to shared locations by default, and never emits real data values in reports.

• **The determination is not yours.** Privacy screens produce evidence and a technical pass/fail; HIPAA/HITRUST de-identification determinations, IRB/privacy sign-off, and disclosure-risk acceptance belong to privacy/compliance/legal. Escalate; do not self-clear a quasi-identifier risk.

Maturity & scaling

• **Assess on a cadence.** Re-run `maturityassess.py` at each review; advance one level deliberately rather than declaring maturity you have not evidenced.

• **Expand by request type, then by program.** Prove a new request type end-to-end (through gates) before offering it broadly. Growth that outruns the gates produces escaped defects and privacy risk.

• **Keep the baseline honest.** The Assess-stage baseline is the yardstick; do not quietly redefine KPIs to look better. A downward trend in rework and defect escape is the credible story to the sponsor.

Deliverables Summary

What the buyer gets: a complete, self-guided package to stand up a **synthetic data sub-function** inside quality engineering as a repeatable, measured service — the operating-model documentation plus five working Python tools.

Documentation

- **Solution Overview** — the sub-function's mission, scope, target model (FHIR/OMOP/native), how it serves AI/QA, the operating model, benefits, and limitations.
- **Requirements Guide** — the six category-wide requirements plus six solution-specific prerequisites (sponsor/charter, pod structure, intake scheme, KPI framework, reference architecture, maturity target), each with how to *verify* and *implement if missing*.
- **Implementation / Operating Model & Roadmap** — the ordered standup following Assess → Design → Generate/Build → Validate → Expand: pod/squad design, intake and prioritization, reference architecture, the KPI catalog, and explicit pass/fail gates with rollback. Every artifact is named (`Capability Charter – Sub-Function – v1`, `Synthetic EHR – Pilot – v1`, etc.).
- **Testing & Validation** — operating-model tests and per-dataset fidelity/coverage/privacy gates, each with an expected pass/fail signal.
- **Troubleshooting** — common standup and dataset failure modes and their resolutions.
- **Best Practices** — durable practices for operating and scaling the sub-function.
- **Capability document template** — the charter/operating-model structure embedded in the implementation guide, ready to fill in.

The 5 Python tools (`SYNFUNC-Scripts.zip`)

All tools are Python 3.10+, standard-library-first, and **safe by default** (dry-run for anything that writes; never write synthetic output to shared locations; never emit real data values).

1. `maturityassess.py` — Capability & maturity self-assessment. Scores the sub-function across dimensions (operating model, engineering, validation/quality, privacy/governance, adoption), computes an overall maturity level, and emits `Maturity Baseline – Sub-Function – v1`.

1. `intakequeue.py` — Intake-queue analyzer & pod-capacity helper. Ingests a request queue, validates the intake schema, applies the value/urgency/effort/risk prioritization scoring model, ranks the queue, and models demand against pod FTE (per-pod utilization vs. a ceiling) to flag over-commitment for the charter and expansion waves.

1. `datasetprofile.py` — Fidelity/coverage report. Profiles a synthetic dataset for structural conformance, in-scope coverage, and (when a reference is supplied) distributional fidelity, with an explicit N/A when no reference exists. Produces the fidelity/coverage section of the validation record.

1. `privacycheck.py` — Privacy/validation check. Screens a synthetic dataset for direct-identifier leakage, exact reference-row copies (memorization), and quasi-identifier/uniqueness risk (k-anonymity signal). Produces technical evidence and a pass/fail signal; defers the disclosure-risk determination to the

privacy officer.

1. `kpirollup.py` — KPI computation & rollup + reference-architecture config validator. Computes the delivery-quality and productivity KPIs from intake + validation records, and (in `--refarch` mode) validates the reference-architecture config for the required tiers, engine interface, gate stage, and the no-real-PHI isolation boundary.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

datasetprofile.py

intakequeue.py

kpirollup.py

maturityassess.py

privacycheck.py